Notes for C++

Section 11 : Classes and Objects

A **class** is a collection of variables (**data)** and **methods** (functions). Usually the data in the classes is part of a logical collection – sometimes called a **record**. For example, a student record may consist of student name, student ID, student email, degree, completed courses, etc.

Example A – setting up a simple date class

```
#include <iostream>
using namespace std;

class date_class {
public:
  int day, month, year;
};

date_class today;

int main() {
  today.day = 23;
  today.month = 11;
  today.year = 2019;
  cout << today.day << "/" << today.month;
  cout << "/" << today.year << endl;
}
```

Important points about this program:

- we set up a **class** called `date_class` with three data items which are day, month, year
- **public** means that the data items can be used anywhere in the program (e.g. inside main)
- we use the class to define an **object** called `today`. One class can be used to create many objects. Sometimes a class is called a **template**.
- we refer to data items inside an object with a dot, for example `today.month`
- data items are normal variables and can be used in the same way as variables

Methods

Classes not only hold variables, they also provide functions to work with the data. These functions are called methods. The methods serve two purposes:

a) they make main shorter and easier to read
b) they make the classes portable (or stand-alone) so they can be placed in a library

Example B – adding methods to the date class

We add two methods to the class – one is to set a date and the other is to display a date.

```
#include <iostream>
using namespace std;

class date_class {
public:
```

```
  int day, month, year;
  void setdate(int d, int m, int y);
  void displaydate();
};

date_class today;

int main() {
  int m;
  today.setdate(23, 11, 2019);
  today.displaydate();
  m = today.month;
  cout << "The month in this date is " << m << endl;
}

void date_class::setdate(int d, int m, int y) {
  day = d;
  month = m;
  year = y;
}

void date_class::displaydate() {
  cout << day << "/" << month << "/" << year << endl;
}
```

Important points about this program:

- sections of main are shorter than Example A (for example, setting the date)
- each method has a **function prototype** inside the class
- Example A shows us that methods are optional (but are often used)
- the methods are functions – they follow all the rules of functions
- the class variables are used inside the methods without a dot, e.g. `day`
- the methods are called with the object name and a dot, e.g. `today.displaydate();`

Methods are **functions** so we follow all the rules about functions. All rules about **parameters** and the use of **return** apply as usual. Each method (function) has a **prototype** but the prototypes sit inside the class. Each method (function) has a **definition** (the code for the function) and these are placed, as usual, after main. However, we need to warn the system that these functions are not simply ordinary functions, they are methods that belong to a particular class. So we write the name of the class before the name of the function with the symbol `::` which means "belongs to". For example, the method `setdate` belongs to the class `date_class` and so we write `void date_class::setdate`

Public and Private

Data and methods inside a class can be defined to be public or private. Private data and methods can only be used within the class itself – they may not be used by main (or any other function). Public data and methods can be used anywhere in the program (inside the class, main, other functions, etc).

You can decide which data and which methods are private and which are public. We will follow a widely-used convention which is:

- make all data private
- make all methods public

The reason for this is that it enables classes to become portable (i.e. can be held in a library and used by many different programs). This would not be possible if the main program had to know about the data in order to use the class.

Example A (above) is ***not*** portable – main has to know all details of the class in order to work
Example B (above) is portable – main does not have to know about the data items in the class

Example C – adding methods to the date class

```
#include <iostream>
using namespace std;

class date_class {
private:
  int day, month, year;
public:
  void setdate(int d, int m, int y);
  int getday();
  int getmonth();
  int getyear();
  void displaydate();
};

date_class today;

int main() {
  int m;
  today.setdate(23, 11, 2019);
  today.displaydate();
  m = today.getmonth();
// m = today.month;  -- causes an error because month is private
  cout << "The month in this date is " << m << endl;
}

void date_class::setdate(int d, int m, int y) {
  day = d;
  month = m;
  year = y;
}

int date_class::getday() {
  return day;
}

int date_class::getmonth() {
  return month;
}

int date_class::getyear() {
  return year;
}

void date_class::displaydate() {
  cout << day << "/" << month << "/" << year << endl;
```

```
}
```

Important points about this program:

- it does the same thing as Example B
- main is not allowed to use the private data that belongs to the class
- main can use the methods which are public
- we have added several "get" methods to the class

Set and Get methods

It is very common to see "set" and "get" methods inside a class. For example, in the program above, `setdate` is used to load the variables and `getday` is used to send the variable `day` back to main. The reason for using "set" and "get" methods is that the data in the class is private and cannot be directly accessed by main. Instructions such as `m = today.month;` are no longer allowed because main is not allowed to directly access the private data. Instead we use `m = today.getmonth();` to send the value of the private variable `month` back to main.

Programmers often have lengthy discussions of which approach is most sensible – is it better to make all data private and then set up "set" and "get" methods? This is the conventional approach but can produce enormously long code if there are many private variables in the class and every variable needs a "set" method and a "get" method. The alternative approach is to make the class data public. This means that main can directly access the class data (see Example A above) but can lead to problems when updating a class, especially if that class is in a library and is used by many programs (more on updating classes later). We will stick to the conventional approach which is:

a) make all class data private and all methods public
b) write "set" and "get" methods as required

Short methods can be defined inside the class

Some methods are very short with only 1 or 2 short lines of code. For example, many "set" and "get" methods are very short. It is a convention that, when a method is very short, it is written directly into the class. In other words, the prototype is replaced by the function definition. It then does not appear in the list of functions (and methods) after main. A method that is defined inside the class is usually written all on one line including the curly brackets.

Some points to remember:

- the method definitions can be written EITHER inside the class OR after main
- if the method definition is inside the class then it replaces the prototype
- any method can be written inside the class but by convention we do not do this for methods that have many lines of code – usually only methods with 1 or 2 lines of code appear in the class

All this is optional. You don't have to do it but you need to be able to recognise it in other people's programs. The only reason for placing method definitions directly into the class is that it makes the overall code shorter.

Classes inside classes

We can define several classes in one program and classes can make use of other classes. For example, we may like to keep a list of people's birthdays where each entry on the list has a name and a birthday and each birthday is a date.

Example D – using classes inside classes

```
#include <iostream>
using namespace std;

class date_class {
private:
  int day, month, year;
public:
  void setdate(int d, int m, int y);
  int getday() { return day; }
  int getmonth() { return month; }
  int getyear() { return year; }
  void displaydate();
};

class birthday_class {
private:
  string name;
  date_class birthday;
public:
  void loaddata(string n, int d, int m, int y);
  void display();
  bool compare(birthday_class other);
};

birthday_class birthday1, birthday2;

int main() {
  birthday1.loaddata("Madonna", 16, 8, 1958);
  birthday1.display();
  birthday2.loaddata("John Tolkien", 3, 1, 1892);
  birthday2.display();
  if (birthday1.compare(birthday2) == true) {
    cout << "They have the same birthday\n";
  } else {
    cout << "They do not have the same birthday\n";
  }
}

//--------------- methods for the date_class ------------
void date_class::setdate(int d, int m, int y) {
  day = d;
  month = m;
  year = y;
}

void date_class::displaydate() {
  cout << day << "/" << month << "/" << year << endl;
}

//--------------- methods for the birthday_class ------------
void birthday_class::loaddata(string n, int d, int m, int y) {
  name = n;
  birthday.setdate(d, m, y);
```

```
}

void birthday_class::display() {
  cout << name << " birthday on ";
  birthday.displaydate();
}

bool birthday_class::compare(birthday_class other) {
  int day1, day2, month1, month2;
// people have the same birthday if day and month are the same
  day1 = birthday.getday();
  month1 = birthday.getmonth();
  day2 = other.birthday.getday();
  month2 = other.birthday.getmonth();
  if ((day1 == day2) && (month1 == month2)) {
    return true;
  }
  return false;
}
```

Important points about this program:

- short methods are defined directly in the class – written as one line
- there are two classes and the `birthday_class` makes use of the `date_class`
- the private rules are still followed, so the birthday_class has to use `birthday.getday()`
- main does not use the `date_class` at all
- note the use of comments to make headings for the method definitions of each class

Arrays of objects

We can set up arrays of objects. For example, write a program that displays all the days that were holidays in New Zealand in a particular year.

Example E – arrays of objects

```
#include <iostream>
using namespace std;

class date_class {
private:
  int day, month, year;
public:
  void setdate(int d, int m, int y);
  void loaddate();
  int getday() { return day; }
  int getmonth() { return month; }
  int getyear() { return year; }
  void displaydate();
};

date_class holiday[20];  // an array of objects
int i, num, year;
```

```
int main() {
  cout << "Enter the year ";
  cin >> year;
  cout << "Enter how many holidays in " << year << ": ";
  cin >> num;
  cout << "Enter all the dates that are holidays in ";
  cout << year << endl;
  for (i = 0; i < num; i++) {
    holiday[i].loaddate();
  }
  cout << "\nList of NZ holidays in " << year << endl;
  for (i = 0; i < num; i++) {
    holiday[i].displaydate();
  }
}

//--------------- methods for the date_class ------------
void date_class::setdate(int d, int m, int y) {
  day = d;
  month = m;
  year = y;
}

void date_class::loaddate() {
  cout << "Enter the date (for example: 18 4 2019) ";
  cin >> day >> month >> year;
}

void date_class::displaydate() {
  cout << day << "/" << month << "/" << year << endl;
}
```

Important points about this program:

- set up an array of objects in the usual way by using the square brackets
- all rules of arrays apply, e.g. index numbers, out of bounds, etc
- note how the array uses the methods, e.g. `holiday[i].loaddate();`

Strings in C++ are objects

When we create a string in C++, we are actually creating an object. For example:
`string msg;`
There is a class called string and we are creating an object called `msg`.

The string class has several methods such as length(). Another interesting method is capacity(). This tells you how many characters can fit into the string. If you load a string with more characters than the capacity, the string will automatically expand to fit the new length of the string.

For example, the string is changed to be longer in the following program. Note how the capacity expands to fit the new string.

```
#include <iostream>
#include <string>
using namespace std;
```

```
string msg;
int len, cap;

int main() {
  msg = "Hello";
  cout << msg << endl;
  len = msg.length();
  cout << "len = " << len << endl;
  cap = msg.capacity();
  cout << "capacity = " << cap << endl;

  msg = "My name is Bilbo Baggins";
  cout << msg << endl;
  len = msg.length();
  cout << "len = " << len << endl;
  cap = msg.capacity();
  cout << "capacity = " << cap << endl;
}
```

Most of the time, it is really useful that a string just expands when needed. However, it comes at a cost. Every time a string has to expand its capacity, the system will spend time doing this. In a time-critical operation, this may cause a problem.

There are a huge number of string methods available in the string class. For more information, see: http://www.cplusplus.com/reference/string/string/

In 159.101 you should be aware that a string in C++ is actually an object. You do not need to know the full string class or the methods (except for length – which we already know about).

File variables in C++ are objects

When we create a file variable in C++, we are actually creating an object. For example:
`fstream datafile;`
There is a class called fstream and we are creating an object called `datafile`.

The fstream class has a lot of private data that is required when accessing a file. For example, it keeps track of which line you have just read from a file and which line will be the next one to be read.

We already know several methods from the fstream class such as:
`datafile.open`
`datafile.close`
`datafile.is_open`

There are many methods available in the fstream class. For more information, see: http://www.cplusplus.com/reference/fstream/fstream/

In 159.101 you should be aware that a file variable in C++ is actually an object of the fstream class. You do not need to know the full fstream class or all the methods – except for those methods that we have used in Section 10 of the notes.